



Engineer to Excel

# SIMATS

---

## ENGINEERING



# SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

# TECHNICAL PRESENTATION

## CSA1407- COMPILER DESIGN

”

# PHASES OF COMPILER

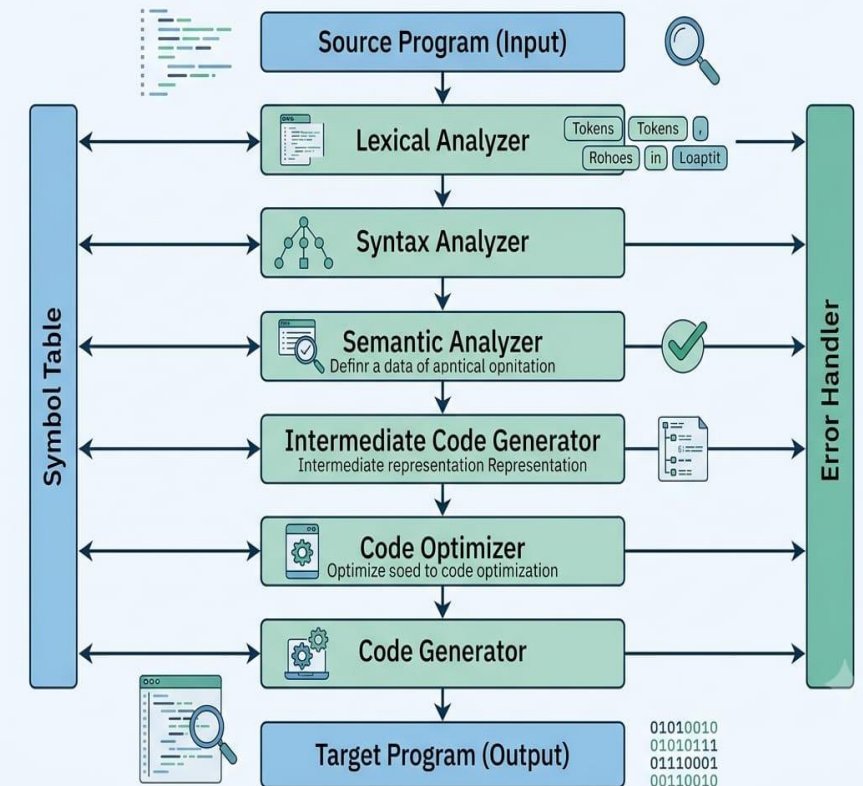
BY

JEROLD SAMUEL J  
192571052  
B.TECH CS BS

# Introduction

- ▶ A compiler converts source code into machine code.
- ▶ The compilation process is divided into different phases.
- ▶ Each phase performs a specific operation.
- ▶ The output of one phase becomes the input for the next phase.
- ▶ All phases work together to generate an executable program.
- ▶ Key Words: Source Code → Compiler → Machine Code

## THE PHASES OF A COMPILER



# Each Phase Performs a Specific Task

- ▶ Every compiler phase has a unique responsibility.
- ▶ It checks and processes the program step by step.
- ▶ Errors are detected during different phases.
- ▶ The compiler improves the program before generating machine code.
- ▶ This modular approach makes compilation efficient and accurate

## **Example:**

- Reading code
- Checking grammar
- Verifying meaning
- Generating machine code

# Includes Lexical, Syntax, Semantic Analysis

## ◆ Lexical Analysis (Scanner)

- ▶ Reads the source code character by character.
- ▶ Converts the code into **tokens** (keywords, identifiers, operators, constants).
- ▶ Removes unnecessary spaces and comments.
- ▶ Detects invalid characters.

## ◆ Syntax Analysis (Parser)

- ▶ Checks whether the tokens follow the programming language grammar.
- ▶ Builds a **Parse Tree**.
- ▶ Detects syntax errors such as missing brackets or semicolons.

# Semantic Analysis

## ◆ Semantic Analysis

- ▶ Checks the meaning of the program.
- ▶ Verifies variable declarations and data types.
- ▶ Ensures operations are logically correct.
- ▶ Detects semantic errors.

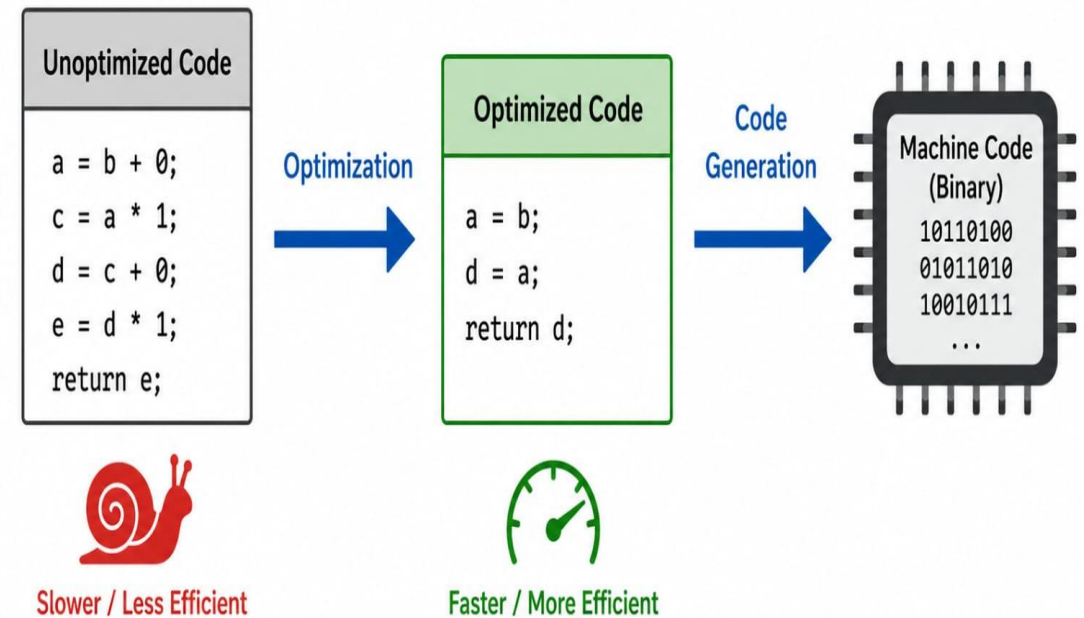
# Code Optimization and Code Generation

## ❖ Code Optimization

- Improves the efficiency of the program.
- Removes unnecessary or duplicate instructions.
- Reduces execution time and memory usage.
- Produces faster and optimized intermediate code.

## ❖ Code Generation

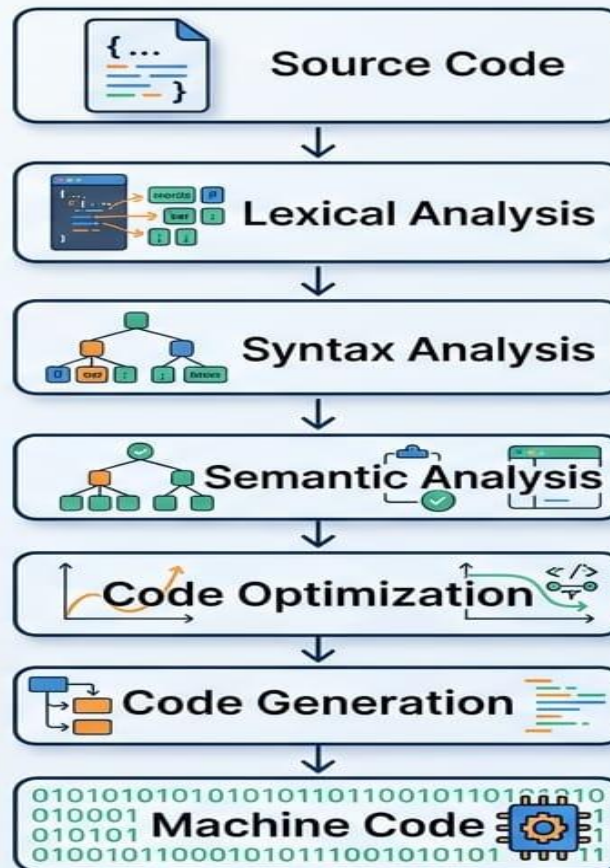
- Converts optimized intermediate code into machine code.
- Generates assembly or executable code.
- Produces the final output that the computer can execute.
- Ensures the generated code is efficient and correct



# Sequential Working

- ▶ The compiler processes the program step by step.
- ▶ Each phase receives the output from the previous phase.
- ▶ Errors are detected and reported during the appropriate phase.
- ▶ The final phase generates machine code.
- ▶ All phases work together to produce an efficient executable program

# Flowchart





*Thank  
you!*